

Exercise 1.1: In-place Reverse

```
import java.util.Arrays;
import java.util.Scanner;

public class Ex11_ReverseInPlace {

    // Reverses the given array in place (modifies the original array)
    public static void reverseInPlace(double[] a) {
        int i = 0, j = a.length - 1;
        while (i < j) {
            double tmp = a[i];
            a[i] = a[j];
            a[j] = tmp;
            i++;
            j--;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();
        double[] a = new double[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter value " + i + ": ");
            a[i] = sc.nextDouble();
        }

        System.out.println("Before: " + Arrays.toString(a));
        reverseInPlace(a);
        System.out.println("After:  " + Arrays.toString(a));

        sc.close();
    }
}
```

Exercise 1.2: Deciding Four Consecutive Numbers

```
import java.util.Scanner;

public class Ex12_ConsecutiveFour {

    public static boolean isConsecutiveFour(int[] values) {
        if (values == null || values.length < 4) return false;

        int run = 1;
        for (int i = 1; i < values.length; i++) {
            if (values[i] == values[i - 1]) {
                run++;
                if (run >= 4) return true;
            } else {
                run = 1;
            }
        }
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter input size: ");
        int n = sc.nextInt();
        int[] a = new int[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter integer " + i + ": ");
            a[i] = sc.nextInt();
        }

        System.out.println(isConsecutiveFour(a)
            ? "Series contains four consecutive equal values."
            : "Series does NOT contain four consecutive equal values.");

        sc.close();
    }
}
```

Exercise 1.3: Longest Run of Equal Values

```
import java.util.Scanner;

public class Ex13_LongestRun {

    public static int longestRun(int[] values) {
        if (values == null || values.length == 0) return 0;

        int best = 1;
        int run = 1;

        for (int i = 1; i < values.length; i++) {
            if (values[i] == values[i - 1]) {
                run++;
                if (run > best) best = run;
            } else {
                run = 1;
            }
        }
        return best;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter input size: ");
        int n = sc.nextInt();
        int[] a = new int[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter integer " + i + ": ");
            a[i] = sc.nextInt();
        }

        System.out.println("Longest run length: " + longestRun(a));
        sc.close();
    }
}
```

Exercise 1.4: Stopwatch to Measure Running Time

StopWatch.java:

```
public class Stopwatch {
    private long startTime;
    private long endTime;

    public Stopwatch() {
        long now = System.currentTimeMillis();
        this.startTime = now;
        this.endTime = now;
    }

    public long getStartTime() { return startTime; }
    public long getEndTime() { return endTime; }

    public void start() {
        startTime = System.currentTimeMillis();
    }

    public void stop() {
        endTime = System.currentTimeMillis();
    }

    public long getElapsedTime() {
        return endTime - startTime;
    }
}
```

Ex14_StopWatchTest.java:

```
import java.util.Random;

public class Ex14_StopWatchTest {

    // Selection sort (in-place)
    public static void selectionSort(int[] a) {
        for (int i = 0; i < a.length - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < a.length; j++) {
                if (a[j] < a[minIdx]) minIdx = j;
            }
            int tmp = a[i];
            a[i] = a[minIdx];
            a[minIdx] = tmp;
        }
    }

    public static void main(String[] args) {
        int n = 100_000;
        int[] a = new int[n];
        Random r = new Random();

        for (int i = 0; i < n; i++) {
            a[i] = r.nextInt();
        }

        Stopwatch sw = new Stopwatch();
        sw.start();
        selectionSort(a);
        sw.stop();

        System.out.println("Selection sort of " + n +
            " integers took " + sw.getElapsedTime() + " ms.");
    }
}
```

Exercise 1.5: Student Records

Student.java:

```
public class Student {
    private String firstName;
    private String lastName;
    private String email;
    private int groupNumber;

    public Student(String firstName, String lastName,
                   String email, int groupNumber)
    {
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.groupNumber = groupNumber;
    }

    public int getGroupNumber() {
        return groupNumber;
    }

    // Compare by group number (smaller groupNumber is "less")
    public boolean less(Student other) {
        if (other == null)
            throw new IllegalArgumentException("other student is null");
        return this.groupNumber < other.groupNumber;
    }

    @Override
    public String toString() {
        return String.format(
            "Student{first='%s', last='%s', email='%s', group=%d}",
            firstName,
            lastName,
            email,
            groupNumber
        );
    }
}
```

Ex15_StudentTest.java:

```
public class Ex15_StudentTest {
    public static void main(String[] args) {
        Student s1 = new Student("Ava", "Li", "ava.li@example.com", 3);
        Student s2 = new Student("Noah", "Wang", "noah.wang@example.com", 1);
        Student s3 = new Student("Mina", "Zhou", "mina.zhou@example.com", 3);

        System.out.println(s1);
        System.out.println(s2);
        System.out.println(s3);

        System.out.println("s1 less than s2? " + s1.less(s2));
        System.out.println("s2 less than s1? " + s2.less(s1));
        System.out.println("s1 less than s3? " + s1.less(s3)); // false
    }
}
```

Exercise 1.6: Book Records

Book.java:

```
public class Book {
    private String title;
    private String author;
    private int year;
    private String isbn;

    public Book(String title, String author, int year, String isbn) {
        this.title = title;
        this.author = author;
        this.year = year;
        this.isbn = isbn;
    }

    public int getYear() {
        return year;
    }

    // Earlier year is "smaller"
    public boolean less(Book other) {
        if (other == null)
            throw new IllegalArgumentException("other book is null");

        return this.year < other.year;
    }

    @Override
    public String toString() {
        return String.format(
            "Book{title='%s', author='%s', year=%d, isbn='%s'}",
            title, author, year, isbn
        );
    }
}
```

Ex16_BookTest.java:

```
public class Ex16_BookTest {
    public static void main(String[] args) {
        Book b1 = new Book("Clean Code",
                           "Robert C. Martin",
                           2008,
                           "978-0132350884");

        Book b2 = new Book("Effective Java",
                           "Joshua Bloch",
                           2018,
                           "978-0134685991");

        Book b3 = new Book("Design Patterns",
                           "Gamma et al.",
                           1994,
                           "978-0201633610");

        Book b4 = new Book("The Pragmatic Programmer",
                           "Hunt & Thomas",
                           1999,
                           "978-0201616224");

        System.out.println(b1);
        System.out.println(b2);
        System.out.println(b3);
        System.out.println(b4);

        System.out.println("b3 less than b4? " + b3.less(b4)); // true
        System.out.println("b2 less than b1? " + b2.less(b1)); // false
        System.out.println("b4 less than b1? " + b4.less(b1)); // true
    }
}
```

Exercise 1.7: Assign Grades

```
import java.util.Scanner;

public class Ex17_AssignGrades {

    public static char gradeFor(int score, int best) {
        if (score >= best - 10) return 'A';
        if (score >= best - 20) return 'B';
        if (score >= best - 30) return 'C';
        if (score >= best - 40) return 'D';
        return 'F';
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of students: ");
        int n = sc.nextInt();
        int[] scores = new int[n];

        System.out.print("Enter " + n + " scores: ");
        int best = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            scores[i] = sc.nextInt();
            if (scores[i] > best) best = scores[i];
        }

        for (int i = 0; i < n; i++) {
            char g = gradeFor(scores[i], best);
            System.out.println("Student " + i + " score is " + scores[i] +
                               " and grade is " + g);
        }

        sc.close();
    }
}
```

Exercise 1.8: Stock Class

UML :

Stock
- symbol: String - name: String - previousClosingPrice: double - currentPrice: double
+ Stock(symbol: String, name: String) + getChangePercent(): double + setPreviousClosingPrice(p: double): void + setCurrentPrice(c: double): void + getSymbol(): String + getName(): String

Stock.java :

```
public class Stock {
    private String symbol;
    private String name;
    private double previousClosingPrice;
    private double currentPrice;

    public Stock(String symbol, String name) {
        this.symbol = symbol;
        this.name = name;
    }

    public String getSymbol() { return symbol; }
    public String getName() { return name; }

    public void setPreviousClosingPrice(double previousClosingPrice) {
        this.previousClosingPrice = previousClosingPrice;
    }

    public void setCurrentPrice(double currentPrice) {
        this.currentPrice = currentPrice;
    }

    public double getChangePercent() {
        if (previousClosingPrice == 0.0) {
            throw new ArithmeticException("previousClosingPrice is 0, so
                                           cannot compute percent change.");
        }
        return (currentPrice - previousClosingPrice) /
            previousClosingPrice * 100.0;
    }
}
```

